AdapterSlots: Warp-Aligned Multi-LoRA Serving via WAR-Gated Kernel Promotion

Shashank Tippanavar
Dept. of Computer Science and Engineering
IIIT Bangalore
Bengaluru, India
shashank.tippanavar@iiitb.ac.in

Vivek Yadav
Dept. of Computer Science and Engineering
IIIT Bangalore
Bengaluru, India
vivek.yadav@iiitb.ac.in

***Abstract*—Serving LLMs with concurrent per-request LoRA adapters is a central production-inference challenge. The dominant batched LoRA kernel, SGMV, suffers operational intensity collapse as the number of active adapters K grows: mixing K adapters can require $K\times$ as many distinct adapter-weight loads and widens the throughput gap between adapter-aligned and mixed batches to 78% at $K{=}16$ (A6000, $N{=}2048$). Nsight Compute profiling shows warp occupancy is unchanged at 8.33% across all mixing conditions, so the gap is not an occupancy effect; the dominant effect is DRAM-bandwidth inflation ($3.07\times$ achieved DRAM throughput, fully-aligned vs. fully-mixed). We present AdapterSlots (AS), a system that attacks this at two levels. At the scheduling layer, a per-adapter alignment buffer holds decode tokens until warp-sized batches form, controlled by Erlang fill-probability deadlines $T^*(k)$ and a PI controller that maintains a closed-loop Warp Alignment Ratio (WAR) target under workload drift (14.3 pp improvement over a static policy), with a Whittle-inspired dispatcher that attains near-oracle WAR at low per-tick cost (an $O(K \log K)$ index sort, measured below 200 µs for $K{\leq}128$). At the kernel layer, WAR-Gated Kernel Promotion (WGKP) uses the Generalized WAR metric $\mathrm{GWAR}(n^*)$ as a runtime gate to promote from SGMV to a fused Triton kernel ($\psi_{\text{fuse}}{=}1.334\times$ at rank 16) or a merged-weight cuBLAS GEMM ($\psi_{\text{promote}}{=}1.436\times$ at rank 32), so that alignment structurally drives throughput (Theorem 3). On a single RTX A6000, the full system achieves $1.525\times$ throughput over vLLM at $K{=}10$, rank=32 (mean of 3 independent runs; CV=0.77%). On LLaMA-13B we observe gains of $1.33\times$–$1.62\times$ at $K{\in}\{4, 8, 16\}$, increasing monotonically with adapter diversity. Adapter-Partitioned Independent Serving (APIS) achieves $2.598\times$ at $K{=}50$ on two RTX A6000 PCIe.*

***Index Terms*—multi-LoRA serving, warp alignment, SGMV, WAR-gated kernel promotion, Erlang deadlines, Whittle-inspired dispatch, APIS**

I. Introduction

Parameter-efficient fine-tuning via LoRA [1] has transformed production LLM serving: a single base model handles hundreds of fine-tuned adapters concurrently [2], [3], each serving a distinct user group or task domain. The dominant kernel for concurrent multi-adapter execution is SGMV [4], which batches each adapter's low-rank matrix products into a segmented GEMM to amortise GPU launch cost. The problem is that SGMV's efficiency degrades sharply as the number of active adapters K grows. On NVIDIA RTX A6000 hardware, at $K{=}16$ the throughput gap between fully adapter-aligned and fully adapter-mixed batches reaches **78%**.

Root cause: DRAM bandwidth inflation. A common assumption blames this gap on GPU warp divergence—threads within a warp taking different execution paths. Occupancy is not the mechanism: `sm__warps_active` is unchanged at **8.33%** across all four mixing conditions (fully aligned through fully mixed). The dominant measured effect is DRAM bandwidth inflation: mixing K adapters forces loading K distinct weight sets, raising achieved DRAM throughput from 10.2% to 31.3% of peak—a $3.07\times$ increase (conditions A through D).

The alignment opportunity. The fix is conceptually simple: hold each adapter's decode tokens in a small buffer until a full GPU warp's worth ($W{=}32$ tokens) accumulates, then dispatch them together. Threads in the warp then operate on the same adapter weight set, substantially reducing the redundant HBM traffic that adapter mixing induces. The challenge is twofold: (1) computing per-adapter waiting deadlines that are short enough to preserve tail latency, and (2) ensuring that a more aligned batch actually runs faster—something prior systems do not guarantee.

Prior systems do not close the loop. Punica [4], S-LoRA [5], and dLoRA [6] all try to group tokens by adapter but then hand the batch to the same SGMV kernel regardless of how well-aligned it is. As a result, alignment correlates with throughput only indirectly—because fewer adapters means less mixing—never because the kernel exploits it.

Our insight: use alignment to select the kernel. AS introduces WAR-Gated Kernel Promotion (WGKP). Instead of running SGMV unconditionally, AS measures how well-aligned the current batch is—using the Generalized WAR metric $\mathrm{GWAR}(n^*)$—and promotes well-aligned segments to progressively faster kernels. This makes alignment a structural determinant of throughput rather than a passive correlate (formalised in Theorem 3; Fig. 1).

Contributions:

- **AlignmentBuffer**: per-adapter queues with Erlang-derived deadlines $T^*(k)$, a PI feedback loop that tracks WAR under workload drift (mean-square stable under an online-estimated gain bound), and a Whittle-inspired dispatcher that keeps overhead under 200 µs at $K{=}128$.
- **WGKP**: a three-level kernel stack (SGMV / Fused Triton / merged cuBLAS GEMM) gated on $\mathrm{GWAR}(n^*)$, with a proof that higher GWAR structurally raises throughput (Theorem 3).

Measured: 1.334× at L2 (rank 16), 1.436× at L3 (rank 32) over SGMV.

- **APIS**: two TP=1 shards instead of PCIe TP=2, shrinking the per-shard scheduling interval from 100.4 ms to 29.3 ms and restoring the alignment budget.
- **Hardware results**: 1.525× on LLaMA-7B at $K{=}10$, 1× RTX A6000 (CV=0.77%); 1.33×–1.62× on LLaMA-13B [7] at $K{\in}\{4, 8, 16\}$; 2.598× via APIS at $K{=}50$ (2× RTX A6000 PCIe).

II. Scheduling Stack: Generating the GWAR Signal

SGMV intensity collapse. For adapter k in a batch of N tokens spread across K adapters, the SGMV operational intensity (tokens per weight-byte loaded) is proportional to $N/(Khr)$, where h is the hidden dimension and r the LoRA rank. As K grows, each adapter gets fewer tokens per batch and the GPU must load more distinct weight sets, collapsing memory efficiency. Hardware profiling (NVIDIA Nsight Compute) shows achieved DRAM throughput rising by over 3× from a fully-aligned to a fully-mixed batch at $K{=}4$. Alignment buffering counteracts this: tokens accumulate per adapter and are dispatched together as full warps.

AlignmentBuffer. AS maintains a per-adapter deque Q_k inside the vLLM scheduler loop; after each iteration's sequence selection, decode tokens for adapter k are appended. Dispatch fires when either condition holds:

$$|Q_k| \geq W \;\text{ or }\; \mathrm{age}(Q_k) \geq T^{(k)}_{\max}, \tag{1}$$

where $W{=}32$ is the warp width: the queue is warp-full, or the per-adapter deadline has expired. The two controllers are alternatives for that deadline, capped alike: Erlang mode sets it to $\min\{T^*(k), T_{\mathrm{SLO}}\}$ (Theorem 1), PI control to $\min\{T^{(t)}_{\max}, T_{\mathrm{SLO}}\}$ for one global $T^{(t)}_{\max}$.

Definition 1 (WAR and GWAR). *Partition each dispatched batch into maximal contiguous same-adapter runs. The Generalized WAR* $\mathrm{GWAR}(n^*)$ *is the fraction of dispatched* tokens *in runs of length* $\geq n^*$*, indexed by the promotion threshold* n^**; this is what the WGKP gate consumes. The Warp Alignment Ratio is the complementary* per-warp *measure: the fraction of the* $\lfloor N/W \rfloor$ *complete warps that are adapter-homogeneous. The denominators differ, so the two values are not directly comparable.*

Erlang fill-probability deadlines. One rate drives the controller: $\hat{\lambda}_k$, the rate at which decode tokens accumulate in Q_k. The EWMA estimator measures it from token inter-arrivals; the inversion, the CASH score and APIS all consume that estimate. A trace's aggregate *request* rate, written λ without hat or subscript, never enters a deadline. Under Poisson token arrivals at rate $\hat{\lambda}_k$ the time to accumulate W warp slots is Erlang-W [8], [9], whose CDF inverts to the deadline achieving a target fill probability.

Theorem 1 (Erlang fill-probability deadline). *Let* $F_{W,\hat{\lambda}_k}$ *be the Erlang-*$W(\hat{\lambda}_k)$ *CDF and* T_{SLO} *a maximum-wait cap. Timing accumulation from an empty* Q_k*, the per-adapter deadline* $T^*(k) = \min\{F^{-1}_{W,\hat{\lambda}_k}(\mathrm{WAR}^*),\ T_{\mathrm{SLO}}\}$ *makes adapter* k *fill a full warp before its deadline with probability exactly* WAR^* *whenever the cap is inactive.*

Proof. Poisson arrivals give i.i.d. exponential inter-arrival times; by convolution the time to accumulate W tokens from an empty queue is Erlang-$(W, \hat{\lambda}_k)$. With the cap inactive, $F(T^*(k)) = \mathrm{WAR}^*$, so $\Pr[T_{\mathrm{fill}} \leq T^*(k)] = \mathrm{WAR}^*$ by definition of the CDF. Strict monotonicity of the Erlang CDF gives uniqueness, and bisection on the regularised incomplete gamma function computes the quantile efficiently. When the cap binds, $T^*(k) < F^{-1}(\mathrm{WAR}^*)$ and the attained fill probability is $F(T_{\mathrm{SLO}}) < \mathrm{WAR}^*$. □

Fill probability, rates, and the cap. Theorem 1 concerns the *probability* that a queue reaches a full warp, not the per-warp WAR of Definition 1: queues expiring at the deadline still dispatch partial runs, so the realised WAR generally differs from WAR^*, which we therefore treat as a control set-point while measuring WAR directly throughout. The scoping matches the implementation: the buffer times each queue from its empty-to-nonempty transition, so every cycle starts at zero and no residual Erlang-$(W{-}s)$ term arises. The Erlang-W CDF depends on rate and deadline only through their product, so attained fill is a function of $\hat{\lambda}_k T^*(k)$ alone: one dimensionless constant (36.638 at $W{=}32$, $\mathrm{WAR}^*{=}0.8$) fixes the family, and a check at one rate is a check at every rate. Fig. 2 runs it at four calibration rates spanning 3.5×, 3.19 down to $0.92\,\mathrm{s}^{-1}$: the product holds at 36.638 and attained fill stays within 0.0033 of WAR^*. These calibrate the inversion rather than being the deployed $\hat{\lambda}_k$; since only the product enters, the result carries to deployment unchanged. At the $K{=}10$ point of §V, 918.5 tok/s over a Zipf-0.9 mix puts the busiest adapter near $3{\times}10^2$ tokens/s: an uncapped quantile of ≈130 ms against the ≈90 ms the PI controller settles on. The other nine have uncapped quantiles of 240 ms to ≈1 s, past the cap, so all sit at T_{SLO}.

PI controller for workload drift. Fixed deadlines go stale when arrival rates shift mid-serving. We close the loop with a PI controller that updates $T_{\max}$ each tick using the alignment gap $e^{(t)} = \mathrm{WAR}^* - \mathrm{WAR}^{(t)}$: $T^{(t+1)}_{\max} = T^{(t)}_{\max} + K_p(e^{(t)} - e^{(t-1)}) + K_i e^{(t)}$, where K_p reacts to the error's rate of change and K_i removes steady-state offset.

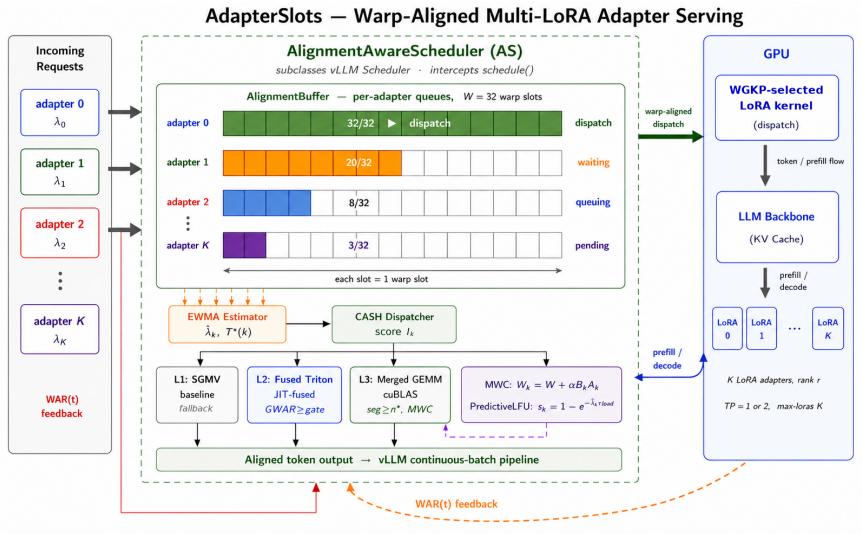

Fig. 1. AdapterSlots system overview. Incoming requests are enqueued in per-adapter deques inside the AlignmentBuffer. The EWMA estimator computes per-adapter token rates $\hat{\lambda}_k$ and Erlang deadlines $T^*(k)$; the CASH dispatcher (CASH score I_k) selects which queues to flush each tick. WAR feedback from the PI controller adjusts $T_{\max}$ under workload drift. WGKP gates dispatched segments into three kernel tiers: L1 SGMV (fallback), L2 fused Triton (GWAR $\geq$ gate), L3 merged cuBLAS GEMM (segment $\geq n^*$ and adapter in MWC). The Merged Weight Cache (MWC) pre-computes $W_k{=}W{+}\alpha B_k A_k$ with PredictiveLFU eviction; aligned output feeds the vLLM continuous-batch pipeline.

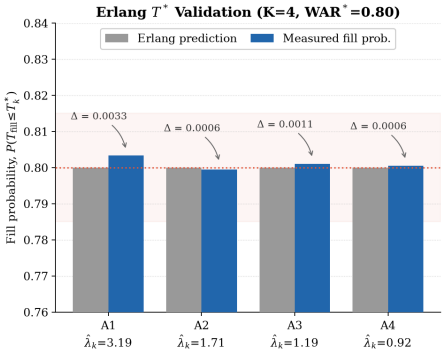

Fig. 2. Erlang T^* validation ($K{=}4$, WAR*$=0.80$, single RTX A6000; calibration rates $\hat{\lambda}_k{=}3.19$–$0.92\,\mathrm{s}^{-1}$, not the deployed values). Grey bars are Erlang predictions; blue bars are measured fill probabilities. Maximum error is 0.0033 across all four adapters (Theorem 1).

Theorem 2 (PI stability)**.** *Let $L{=}f'(T^\circ){>}0$ be the local slope of $f: T_{\max} \mapsto \mathbb{E}[\mathrm{WAR} \mid T_{\max}]$ at the loop's operating point T°, distinct from the Erlang deadline $T^*(k)$. Under this linearisation the closed loop is locally asymptotically stable if and only if*

$$K_p > 0, \qquad K_i > 0, \qquad 2K_p + K_i < 2/L,$$

and bounded workload noise then keeps the WAR deviation bounded in mean square. We estimate L online from finite differences of the WAR signal and clip the gains into this region at every step.

Proof Sketch. Put $x_t{=}T_{\max}^{(t)}{-}T^\circ$ and write $a{=}LK_p$, $b{=}LK_i$. The linearisation gives $e^{(t)}{=}{-}Lx_t$, so the update becomes $x_{t+1}{=}(1{-}a{-}b)x_t + a\,x_{t-1}$, with characteristic polynomial $z^2{-}(1{-}a{-}b)z{-}a$. Jury's criterion for a monic $z^2{+}a_1z{+}a_0$ requires $|a_0|{<}1$, $P(1){>}0$ and $P(-1){>}0$, which here read $a{<}1$, $b{>}0$ and $2a{+}b{<}2$; the last implies the first when $b{>}0$, giving the stated region after dividing by L. A bounded-noise Lyapunov argument then gives mean-square boundedness around the set-point. □

On A6000 traces with a sudden $3\times$ rate shift the loop holds the target, $+14.3$ pp over a fixed deadline (post-drift WAR 0.800 vs. 0.657, 20,000 ticks, $K{=}4$), at under $25\,\mu$s per tick.

Whittle-inspired dispatch (CASH). At each scheduling tick, the dispatcher must decide which queues to flush. This is an instance of the Restless Multi-Armed Bandit (RMAB) problem [10], which is generally intractable to solve optimally. AS uses the **CASH** (Cross-Iteration Accumulation with Speculative Holdback) dispatcher, which assigns each adapter queue k a priority score inspired by the Whittle index framework:

$$I_k(s_k) = \hat{\lambda}_k \cdot s_k \cdot P_k(s_k), \tag{2}$$

where $s_k = |Q_k|/W$ is how full the queue is and $P_k(s_k)$ is the Erlang-derived probability it will fill completely before its deadline. The score follows the monotone dispatch structure an index rule needs, but the actual Whittle index requires solving a per-arm Lagrangian MDP; (2) is a closed-form stand-in for it. Indices are computed in $O(K)$ and queues dispatched in decreasing index order via an $O(K \log K)$ sort; the per-tick cost is empirically near-linear over the measured range and stays below $200\,\mu$s for $K{\leq}128$.

Proposition 1 (Near-oracle dispatch, empirical)**.** *Our reference* oracle *is an offline clairvoyant dynamic program that sees the future arrival sequence and maximises WAR*

over a rolling H=20-tick horizon, tractable only for $K \leq 4$. Empirically CASH attains WAR close to this finite-horizon oracle on both Zipf (α=0.9) and adversarial arrival patterns (K=4, A6000): on Zipf workloads all policies converge to the alignment ceiling (WAR 0.805); on adversarial workloads CASH reaches WAR 0.823 vs. 0.819 for both oracle and threshold (Fig. 4a). The oracle is optimal only within its 20-tick lookahead, so the 0.4 pp is not evidence that CASH beats it. That per-tick cost is negligible relative to τ_{iter}=30 ms.

Proof Sketch. Each adapter queue is an arm of an RMAB. The score in (2) is motivated by the Lagrangian relaxation of the per-arm value function rather than obtained from it exactly. Raising the dispatch subsidy shrinks the set of states in which dispatching is optimal; that monotone structure motivates an index rule, though we do not claim indexability, which would require the per-arm MDP. The *exact* index is asymptotically optimal under the condition Weber and Weiss give (§VII); since (2) is not that index, near-oracle behaviour is an *empirical* claim. □

III. WAR-Gated Kernel Promotion (WGKP)

The scheduling stack raises WAR and GWAR, but prior systems route every batch through the same SGMV kernel regardless of alignment quality, leaving most of that benefit unused. WGKP makes the kernel choice a function of how well-aligned the current batch is, measured by $\text{GWAR}(n^*)$ (n^*=8 by default). We keep the two roles of n^* separate: n_2 is the run-length threshold inside GWAR, $n_3 \geq n_2$ the segment-size gate for L3; by default $n_2{=}n_3{=}n^*{=}8$.

Three-level kernel hierarchy. L1 (SGMV): always available; two HBM-bound matrix launches with an intermediate tensor. **L2 (Fused Triton [11]):** a single JIT-compiled kernel that tiles partial results in on-chip SRAM, eliminating that tensor's HBM round-trip; 1.334× over SGMV (at n=32, rank=16; rank=32 yields higher speedup). L2 sits behind a GWAR gate (Fig. 1) that we run open in the evaluated configuration, so every dispatched segment takes at least L2. **L3 (Merged cuBLAS GEMM):** the Merged Weight Cache (MWC) asynchronously pre-computes $W_k{=}W{+}\alpha B_k A_k$, turning two low-rank matmuls into one full-weight BLAS call; 1.436× over SGMV (at n=8, rank=32); activates when segment size $\geq n^*$ and adapter k is in MWC. Fig. 3 reports both speedups against segment size and rank. The ladder is rank-dependent: at rank 32 ψ_{promote} exceeds ψ_{fuse} at every measured segment size, whereas at ranks 16 and 64 the ordering is not uniform across segment sizes. Our promotion policy is not rank-aware, so where L3 falls below L2 it can give up some kernel efficiency; the rank-16 cross-validation of §V-B therefore draws its gain primarily from the scheduling stack and L2.

Observation (GWAR monotonicity). $\text{GWAR}(n^*)$ is monotonically non-increasing in n^*: a stricter run-length threshold $n_2^* \geq n_1^*$ cannot admit more tokens, so $\text{GWAR}(n_1^*) \geq \text{GWAR}(n_2^*)$.

Theorem 3 (Alignment structurally increases throughput under WGKP)**.** *Let g_3 be the L3-eligible token fraction—tokens in same-adapter runs of length $\geq n_3$ whose adapter is resident in the MWC—and let $g_2{=}\text{GWAR}(n_2){-}g_3 \geq 0$ be the remaining L2-eligible fraction, with $n_3 \geq n_2$. Under WGKP with $\psi_{\text{promote}} > \psi_{\text{fuse}} > 1$, throughput Φ is strictly increasing in g_3 at fixed $\text{GWAR}(n_2)$ and strictly increasing in $\text{GWAR}(n_2)$ at fixed g_3:*

$$\frac{\partial \Phi}{\partial g_3} > 0, \qquad \frac{\partial \Phi}{\partial\, \text{GWAR}(n_2)} > 0.$$

Within the modelled cost structure ($\psi_{\text{promote}} > \psi_{\text{fuse}} > 1$, which our rank-32 measurements satisfy at every segment size; the ladder is rank-dependent, as noted above), the relationship is structural rather than merely correlational: WGKP gates kernel selection directly on GWAR, so raising alignment activates faster kernels by construction. We use "causal" in this design-internal sense—an intervention on GWAR changes which kernel executes—not as a claim that holds for arbitrary hardware or workloads.

Proof Sketch. Tokens split into g_3 at L3, g_2 at L2 and $1{-}g_2{-}g_3$ at L1, so $\Phi \propto [(1{-}g_2{-}g_3) + g_2/\psi_{\text{fuse}} + g_3/\psi_{\text{promote}}]^{-1}$. Substituting $g_2{=}\text{GWAR}(n_2){-}g_3$ and raising g_3 by δ at fixed $\text{GWAR}(n_2)$ shifts tokens from L2 to L3 and decreases the bracket by $\delta(1/\psi_{\text{fuse}} - 1/\psi_{\text{promote}}) > 0$; raising $\text{GWAR}(n_2)$ by δ at fixed g_3 shifts tokens from L1 to L2 and decreases it by $\delta(1 - 1/\psi_{\text{fuse}}) > 0$. Both derivatives are therefore positive. The boundary case $n_3{=}n_2$ used by default is covered: every L2-eligible run is then also long enough for L3, so g_2 reduces to exactly those aligned tokens whose adapter is outside the MWC, and the same two displacements apply. □

Theorem 3 is what separates AS from all prior LoRA-serving systems: alignment state is not a heuristic but a structural determinant of which hardware path executes. Both derivatives describe the gated policy; in the configuration we evaluate the L2 gate is open, so $g_1{=}0$, $g_2{=}1{-}g_3$, and the model collapses to $\Phi \propto [(1{-}g_3) + g_3/\psi_{\text{promote}}]^{-1}$: the $\text{GWAR}(n_2)$ derivative no longer applies and all realised gain flows through $\partial\Phi/\partial g_3$, which the L3 gate ties to run-length alignment. The r=0.997 correlation (Fig. 7) is an empirical check on that trend; direction comes from the WGKP gating rule, not from the correlation.

Merged Weight Cache and prefetch. Merging adapter weights into W_k takes about 17.5 ms per adapter. The Alignment-Aware Prefetch (AAP) trigger hides it by starting the merge asynchronously as soon as a queue is half-full: across the K=10 mix the remaining half-warp takes 56–446 ms to arrive, a 3.2–25.5× budget against the merge.

MWC memory budget. LoRA targets all seven linear projections, but the MWC merges only the four attention projections (Q, K, V, O); MLP projections always execute on L1/L2. A merged W_k is materialised *per layer* across all 32 transformer layers, so Δ_k occupies $4{\times}32{\times}4096^2{\times}2$ B ≈ 4.29 GB per adapter in FP16. The MWC therefore holds only the K_{hot} hottest adapters: on the 48 GB A6000 we set

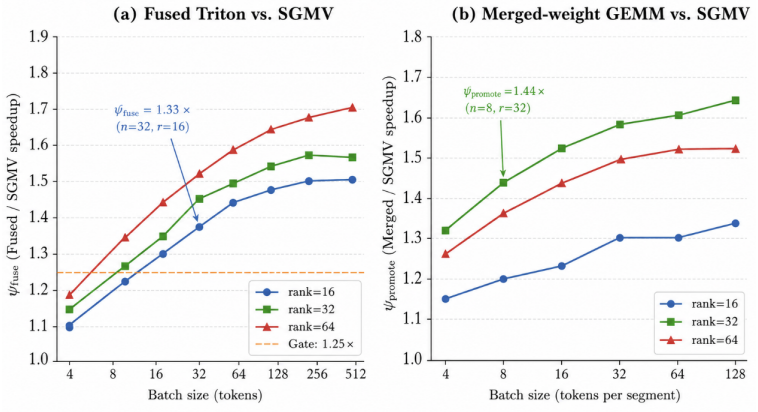

Fig. 3. Measured kernel speedups on RTX A6000 TP=1. **(a)** Fused Triton vs. SGMV: ψ_{fuse} clears the 1.25× gate at $n \geq 32$ (rank 16) and $n \geq 16$ (rank$\geq$32); 1.334× at n=32, rank=16. **(b)** Merged cuBLAS vs. SGMV: ψ_{promote} reaches 1.436× at n=8, rank=32, where the MWC amortises the 17.5 ms merge cost.

K_{hot}=5 ($\approx$ 21.5 GB), leaving the rest for base weights and the paged KV cache; K_{hot}=10 (42.9 GB) would not co-reside with it. Adapters outside the MWC fall back to L2/L1, so the merged cache competes with, and is bounded by, KV-cache capacity. The footprint scales as $4Lh^2 \times 2$ B, so on LLaMA-13B (L=40, h=5120) it is 8.39 GB per adapter and at most one merged adapter co-resides with base weights and KV cache; the 13B gains of §V-C therefore come predominantly from the scheduling stack and the universally-applied L2 kernel rather than from L3.

Amdahl ceiling. On LLaMA-7B with rank=32 on all linear projections, LoRA computation is about 40% of per-iteration time, so Amdahl's law caps the speedup from fully eliminating it at 1.67×; the measured 1.525× reaches **91.5%** of that ceiling.

IV. Adapter-Partitioned Independent Serving (APIS)

The PCIe scheduling barrier. Naively scaling to two GPUs with tensor-parallelism (TP=2) over PCIe inflates the per-decode-step time from 29.3 ms to 100.4 ms, because every step must wait for an allreduce synchronisation across the PCIe bus. This makes any alignment deadline shorter than 100 ms invisible to the scheduler—effectively disabling all alignment benefit. On this dual-A6000 *PCIe* testbed the barrier is an interconnect constraint, not a software limitation, and is specific to PCIe-class allreduce: on NVLink/NVSwitch fabrics or newer datacenter GPUs the allreduce is far cheaper, so TP=2 need not exhibit it and APIS should help less. We scope this claim to PCIe-connected multi-GPU nodes.

APIS topology. AS resolves this by replacing TP=2 with two independent TP=1 servers. An adapter-affinity router sends each request to whichever shard holds its adapter. Since Zipf routing makes an equal-*count* split badly unbalanced in traffic, adapters are assigned by rate-sorted round-robin—order by $\hat{\lambda}_k$ descending, adapter i to shard $i \bmod n$—equalising load rather than adapter count, refreshed from the EWMA rates every 30 s. Each shard now runs at the full 29.3 ms granularity, restoring the buffer's ability to fill warps efficiently. Two TP=1 servers without AS scheduling give only 1.06×; the rest comes from per-shard AS under the higher adapter diversity of each half-pool.

Preemption safety. Dropping a preempted sequence's buffered tokens would throw away alignment progress already accumulated, so we keep a shadow queue Q'_k per adapter: those tokens stay there and return to the head of Q_k on resumption. WAR loss from preemptions is negligible at typical production rates.

PredictiveLFU. When adapters exceed GPU VRAM the system must evict some, and LRU eviction ignores arrival rates, discarding frequently-used adapters at the wrong moment. AS's PredictiveLFU instead scores each adapter by the probability it is needed again before it could be reloaded—an exponential formula in the EWMA rate $\hat{\lambda}_k$ the system already tracks. The least likely is evicted. This costs one score and comparison per eviction on state the estimator already keeps and, replayed over recorded arrival traces, gives a +4.5 pp cache hit-rate improvement over LRU at large adapter pools.

V. Evaluation

Experimental setup. Hardware: NVIDIA RTX A6000 (48 GB, single and dual PCIe). **Primary workload**: LLaMA-7B [7] (FP16), ShareGPT prompts, Zipf-α=0.9 adapter routing, λ=7 req/s, 5,000 requests. Claims C1 and C2 (Table IV) are 3 independent replications (distinct seeds); C3 is a six-point $T_{\max}$ sweep on one trace. **Adapters**: rank-32 LoRA on all seven linear projections (Q, K, V, O, gate, up, down) unless a table states otherwise. **System settings**: W=32, n_2=n_3=8, WAR*=0.8, K_{hot}=5 (MWC scoped to Q/K/V/O), T_{SLO}=200 ms, APIS rebalance 30 s. **LLaMA-13B workload**: same harness, vLLM 0.6.3 for both arms, production stack. **Baselines** (K=10): vLLM 0.5.0, pinned to the published Punica/S-LoRA environments, Punica [4], S-LoRA [5] and dLoRA [6]. Sarathi-Serve [12] chunks prefill, orthogonal to adapter scheduling, and enters at K=4 instead. All baselines run on identical hardware, adapter sets and request traces. Throughput, latency and kernel numbers are hardware measure-

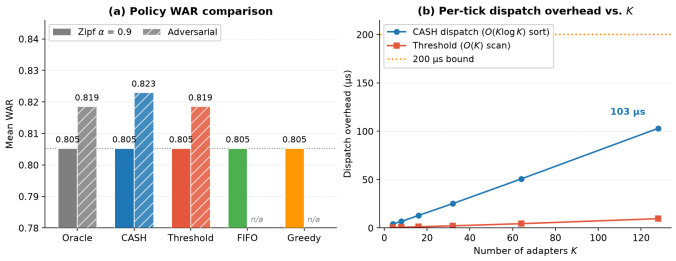

Fig. 4. CASH (Whittle-inspired) dispatcher vs. oracle and threshold baselines (K=4, Zipf α=0.9, RTX A6000; Proposition 1). **(a)** Mean WAR for five policies on Zipf (solid); the adversarial workload (hatched) covers Oracle, CASH and Threshold only (FIFO and Greedy n/a). All five reach the Zipf ceiling (WAR 0.805); on adversarial patterns CASH reaches 0.823 against 0.819 for Oracle and Threshold. **(b)** Per-tick dispatch overhead vs. adapter count K. CASH's $O(K \log K)$ index sort reaches 103 µs at K=128, inside the 200 µs bound and empirically near-linear here.

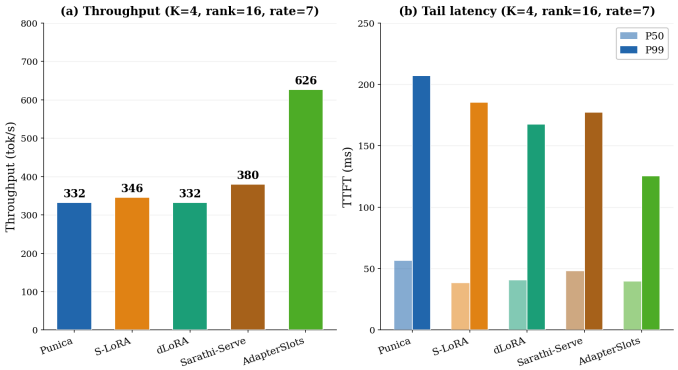

Fig. 5. Throughput and tail latency vs. four SOTA baselines at K=4, rank 16, λ=7 req/s, 500 prompts, single RTX A6000. AS reaches 626.3 tok/s (1.65× over Sarathi-Serve, 1.81–1.89× over Punica/S-LoRA/dLoRA) while posting the lowest P99 TTFT and a P50 within 1.2 ms of the best baseline.

ments; the PredictiveLFU comparison (§IV) is a replay over recorded arrival traces and is reported separately from them.

A. Component Ablation

Three orthogonal gain sources emerge from Table I. (The "Full" row and Table II are one development run, 918.5 vs. 608.7 tok/s; Table IV gives the 3-rep mean 1.525×, CV=0.77%.) The scheduling stack (+*Buffer* through +*CASH*) already pushes WAR up substantially without touching the kernel; CASH's own share is +11 pp WAR over the Erlang-only stack. The K=4 tie in Fig. 4a is low-contention: four queues and W=32 let every policy reach the ceiling. Adding the fused Triton kernel (+*Fused*) speeds up all dispatches uniformly—alignment level does not matter here. WGKP (+*L3 GEMM*) is different: its gain is gated on GWAR, so the kernel change alone cannot unlock it; the batch must be aligned enough to reach the promotion threshold. Each row is an independent end-to-end run: kernel latency feeds back into iteration timing and hence into what the scheduler can accumulate, so WAR and GWAR are measured per configuration, not held fixed.

Operating envelope. In untuned configurations at $K \leq 8$ AS can underperform vLLM: sparse queues yield too few long aligned runs for frequent L3 promotion. The binding constraint is the promotion gate, not the L2 gate, which is open throughout. Macro-batching accumulates more tokens per step, raising the promotion-eligible fraction and restoring positive gain.

B. SOTA Comparison

Table II reports the K=10 primary SOTA comparison; Fig. 5 and Table III report an independent cross-validation at K=4.

Table II shows AS outperforming all baselines at K=10 on both throughput and latency. The lower TTFT may seem surprising given that AS holds tokens for up to T_{SLO}=200 ms; we attribute it to baseline saturation and a backlog we infer rather than instrument (§VI).

C. LLaMA-13B Throughput Scaling

Table V and Fig. 6 report measured throughput on LLaMA-13B with the production deployment stack (vLLM 0.6.3, FP16).

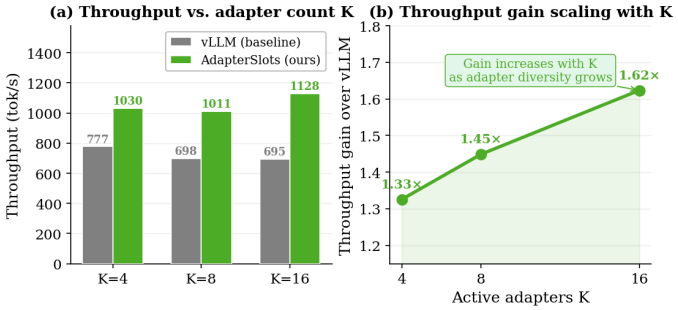

Fig. 6. Final measured throughput on LLaMA-13B at $K \in \{4, 8, 16\}$ (single RTX A6000, production stack). **(a)** Absolute throughput of vLLM and AS. **(b)** Speedup over vLLM increases monotonically with K (1.33× at K=4 to 1.62× at K=16), consistent with Theorem 3.

The highest single-GPU speedup, 1.62×, is at K=16, with gains rising monotonically in adapter count.

D. Primary Claims

A CV of 0.77% across three independent runs shows C1 is reproducible. C3's near-unity GWAR–throughput r is consistent with the mechanism Theorem 3 formalises, the WGKP gate selecting faster kernels as GWAR rises. C2 is measured against vLLM TP=2 on the same two GPUs (1193.5 vs. 459.4 tok/s, 3-rep means) and needs both ingredients: the TP=1 topology alone yields only 1.06×, but it re-opens the scheduling window that lets per-shard AS do the heavy lifting.

VI. Discussion

When gains are largest. AS works best when alignment queues fill regularly: roughly $K \in [4, 16]$ adapters, rank≥16, moderate-to-high request rates, all-linear targets. At very low arrival rates queues rarely fill and buffering adds latency without gain. Our sweeps stop at K=50; beyond that we expect GWAR to fall below the L3 gate, leaving the fused kernel and PredictiveLFU efficiency as the remaining gains.

Why latency improves despite buffering. AS adds alignment buffering up to the T_{SLO}=200 ms cap (near 90 ms on the busiest adapters), yet delivers lower TTFT than vLLM here. We

TABLE I

COMPONENT ABLATION (K=10, RANK=32, λ=7 REQ/S, RTX A6000). WAR IS PER-WARP, GWAR=GWAR(8) (DEF. 1), DIFFERENT DENOMINATORS; PROMO=L3-ELIGIBLE FRACTION.

Stack	Mechanism	Gain	WAR	GWAR	Promo
Baseline	vLLM	1.000×	0.00	0.00	—
+Buffer	Alignment buffer	1.049×	0.32	0.06	—
+Erlang	Per-adapter T^*	1.089×	0.50	0.18	—
+CASH	CASH dispatch	1.149×	0.61	0.28	—
+Fused	Fused Triton (L2)	1.228×	0.63	0.29	—
+L3 GEMM	Merged cuBLAS (L3)	1.344×	0.64	0.39	24.3%
+Macro	Cross-iter batching	1.441×	0.68	0.42	35.1%
Full	CASH n^*=8	**1.509**×	0.71	0.45	37.4%

TABLE II

SOTA THROUGHPUT COMPARISON AT λ=7 REQ/S, K=10, RANK=32, ALL-LINEAR LORA (RTX A6000). SARATHI-SERVE EXCLUDED AT K=10 (CHUNKED-PREFILL ORTHOGONAL TO ADAPTER SCHEDULING); INCLUDED AT K=4.

System	Throughput (tok/s)	vs. vLLM	TTFT P50 (ms)	TTFT P99 (ms)
vLLM	608.7	1.00×	1330	1823
Punica	634.3	1.04×	1280	1767
S-LoRA	641.6	1.05×	1250	1703
dLoRA	623.3	1.02×	1287	1795
AS (ours)	**918.5**	**1.51**×	**864**	**1208**

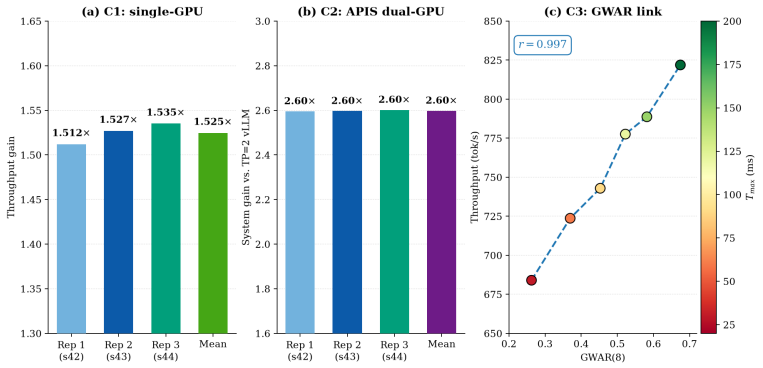

Fig. 7. Three primary claims, full statistical reporting. **(C1)** 1.525× single-GPU at K=10, rank=32 (3-rep mean; CV=0.77%). **(C2)** 2.598× APIS gain over vLLM TP=2 at K=50, 2× RTX A6000 PCIe (3-rep mean; CV=0.13%). **(C3)** GWAR(8)-throughput Pearson r=0.997 over a six-point $T_{\max}$ sweep (30–200 ms). All data from real hardware runs.

TABLE III

CROSS-VALIDATION AT K=4, RANK 16, λ=7 REQ/S (RTX A6000).

System	Tput (tok/s)	TTFT P50	TTFT P99
Punica	331.8	56.5 ms	207.1 ms
S-LoRA	346.0	38.4 ms	185.5 ms
dLoRA	332.1	40.5 ms	167.8 ms
Sarathi-Serve	379.7	48.1 ms	177.3 ms
AS	**626.3**	**39.6 ms**	**125.5 ms**

attribute this to baseline saturation: at λ=7 req/s vLLM runs close to capacity, so a growing backlog dominates its TTFT while AS stays below it. We instrument TTFT, not queue depth, so this is an attribution consistent with the measured throughput, not an observed backlog.

Latency metrics. We report output throughput and TTFT. For fixed output length N, per-request time is TTFT + $(N-1)$ TPOT, so throughput depends on both, not on $1/\text{TPOT}$ alone; AS improves both, so the gain is not a TPOT effect by itself. Full latency distributions are left to an extended version.

Scope of the analytical results. Theorems 1–3 and Proposition 1 are conditional properties under stated assumptions: Poisson per-adapter arrivals, locally stationary rates, and the modelled kernel-cost structure, to be read as such rather than as universal laws.

Generalizability. Our measurements are confined to a single GPU class (RTX A6000, 48 GB), a PCIe interconnect, FP16, and ShareGPT traffic with Zipf-α=0.9 routing at one arrival rate (λ=7 req/s), moderate adapter counts, and the vLLM 0.5.0/0.6.3 stacks. The DRAM-inflation diagnosis and the L2/L3 crossovers depend on the A6000's memory-bandwidth-to-compute ratio, so on A100/H100-class parts the crossovers, and hence the promotion thresholds, need re-calibrating before

TABLE IV
PRIMARY CLAIMS (RTX A6000). C1 AND C2 ARE MEANS OVER 3 INDEPENDENT RUNS; C3 IS A SIX-POINT $T_{\max}$ SWEEP ON ONE TRACE, SO ITS r SUMMARISES A DESIGNED SWEEP, NOT INDEPENDENT REPLICATIONS.

Claim	Mean	Min	Max	CV
C1: Throughput gain, K=10, rank=32, single GPU	1.525×	1.512×	1.535×	0.77%
C2: APIS gain vs. vLLM TP=2, K=50, 2× RTX A6000 PCIe	2.598×	2.596×	2.602×	0.13%
C3: GWAR(8)–throughput Pearson r (n=6)	0.997	—	—	sweep

TABLE V
LLAMA-13B THROUGHPUT (SINGLE RTX A6000, PRODUCTION STACK).

Config.	vLLM (tok/s)	AS (tok/s)	Speedup
K=4	776.98	1030.1	**1.33**×
K=8	698.1	1011.4	**1.45**×
K=16	695.0	1128.3	**1.62**×

the speedups carry. The PCIe barrier motivating APIS is likewise interconnect-specific (§IV), so the gains are workload- and platform-specific. Broader rate sweeps, newer stacks (vLLM V1, SGLang), NVLink/NVSwitch multi-GPU and much larger adapter counts are future work. Three replications make the throughput *means* tight (CV$\leq$0.77%), but tail percentiles are indicative rather than definitive.

Memory and limitations. APIS roughly doubles total model memory against TP=2, since each TP=1 shard holds a full copy where a TP=2 rank holds half: manageable for LLaMA-7B but needing high-memory GPUs for larger models. On-demand Level-3 promotion is profitable only for high-frequency adapters, the async prefetch trigger extending this to moderate rates; at very low rates AS falls back to standard vLLM scheduling.

VII. RELATED WORK AND CONCLUSION

LLM serving foundations. Orca [3] introduced iteration-level continuous batching, vLLM [2] PagedAttention for memory-efficient KV management, and SGLang [13] RadixAttention for prefix reuse. AlpaServe [14] and Megatron-LM [15] address model parallelism; APIS builds on the same TP=1 topology insight.

Multi-LoRA serving. Punica [4] and S-LoRA [5] are the primary multi-LoRA serving systems; neither uses alignment state as a kernel-selection signal. dLoRA [6] merges weights at request granularity without GWAR gating. CaraServe [16] offloads adapter weights to CPU DRAM without addressing alignment. Zhou et al. [17] optimise the SGMV operator itself (1.30–1.46×), complementary to WGKP, which leaves the operator fixed and uses runtime alignment to choose *which* tier runs. Concurrent work: AdaFuse [18], InfiniLoRA [19], LoRAFusion [20]. None apply GWAR-gated kernel promotion. Punica reports a negligible same-adapter/mixed-adapter SGMV gap where we measure 78% at K=16, N=2048; the settings differ in GPU class (A6000 GDDR6 vs. A100 HBM2e), batch size, adapter count and SGMV version, and our figure is taken where the intensity collapse of §II is sharpest. We read these as different operating points, scoped to this GPU class.

Kernel optimisation. FlashAttention [21] and FlashInfer [22] optimise the attention tier and Sarathi-Serve [12] prefill chunking; AS targets the orthogonal LoRA-projection tier in decode. Triton [11] compiles our L2 kernel.

Scheduling theory. The Whittle index [23] is well-studied in restless bandit scheduling; Weber and Weiss [24] proved its asymptotic optimality as $K \to \infty$ under a global-stability condition on the fluid limit. Theorem 1 builds on Erlang's classical formula [8] and standard queueing results [9]; operational intensity follows the Roofline model [25].

Conclusion. We presented AdapterSlots (AS), which tackles SGMV operational intensity collapse from both the scheduling and kernel sides. The scheduling stack combines analytically-derived per-adapter deadlines (Theorem 1), closed-loop PI control (Theorem 2) and a Whittle-inspired dispatcher with near-oracle WAR (Proposition 1); WGKP converts that alignment signal into a structural throughput advantage (Theorem 3). APIS removes the PCIe scaling barrier and PredictiveLFU maintains cache efficiency at large adapter counts. Every throughput, latency and kernel result here is measured on real hardware: 1.525× on a single GPU (LLaMA-7B), 1.33×–1.62× with adapter diversity (LLaMA-13B), and 2.598× on two GPUs via APIS; the PredictiveLFU hit rate is the one replayed number.

ACKNOWLEDGMENT

GPU profiling used NVIDIA Nsight Compute and Nsight Systems. Code: https://github.com/vac014/AdapterSlots.

REFERENCES

[1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.

[2] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. SOSP*, 2023.

[3] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for transformer-based generative models," in *Proc. OSDI*, 2022.

[4] L. Chen, Z. Ye, Y. Wu, D. Zhuo, L. Ceze, and A. Krishnamurthy, "Punica: Multi-tenant LoRA serving," in *Proc. MLSys*, 2024.

[5] Y. Sheng, S. Cao, D. Li, C. Hooper, N. Lee, S. Yang, C. Chou, B. Zhu, L. Zheng, K. Keutzer, J. E. Gonzalez, and I. Stoica, "S-LoRA: Serving thousands of concurrent LoRA adapters," in *Proc. MLSys*, 2024.

[6] B. Wu, R. Zhu, Z. Zhang, P. Sun, X. Liu, and X. Jin, "dLoRA: Dynamically orchestrating requests and adapters for LoRA LLM serving," in *Proc. OSDI*, 2024.

[7] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv:2302.13971*, 2023.

[8] A. K. Erlang, "Solution of some problems in the theory of probabilities of significance in automatic telephone exchanges," *Elektroteknikeren*, vol. 13, 1917.

[9] L. Kleinrock, *Queueing Systems, Volume I: Theory*. Wiley-Interscience, 1975.

[10] C. H. Papadimitriou and J. N. Tsitsiklis, "The complexity of optimal queuing network control," in *Proc. Conference on Structure in Complexity Theory*, 1994.

[11] P. Tillet, H.-T. Kung, and D. Cox, "Triton: An intermediate language and compiler for tiled neural network computations," in *Proc. MAPL*, 2019.

[12] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve," in *Proc. OSDI*, 2024.

[13] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution of structured language model programs," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2024.

[14] Z. Li, L. Zheng, Y. Zhong, V. Liu, Y. Sheng, X. Jin, Y. Huang, Z. Chen, H. Zhang, J. E. Gonzalez, and I. Stoica, "AlpaServe: Statistical multiplexing with model parallelism for deep learning serving," in *Proc. OSDI*, 2023.

[15] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training multi-billion parameter language models using model parallelism," in *arXiv:1909.08053*, 2019.

[16] S. Li, H. Lu, T. Wu, M. Yu, Q. Weng, X. Chen, Y. Shan, B. Yuan, and W. Wang, "CaraServe: CPU-assisted and rank-aware LoRA serving for generative LLM inference," *arXiv:2401.11240*, 2024.

[17] C. Zhou, Y. Zhou, S. Zhang, Y. Wang, and Z. Liu, "Dynamic operator optimization for efficient multi-tenant LoRA model serving," *Proc. AAAI Conf. Artif. Intell.*, vol. 39, no. 21, pp. 22 910–22 918, 2025.

[18] Q. Li, R. Kong, Y. Li, H. Cai, S. Wang, L. Kong, G. Chen, and D. Yin, "AdaFuse: Accelerating dynamic adapter inference via token-level pre-gating and fused kernel optimization," *Proc. AAAI Conf. Artif. Intell.*, vol. 40, no. 37, pp. 31 680–31 688, 2026.

[19] H. Chen, L. Ruan, Z. Xu, Y. Li, X. Chen, J. Leng, B. He, M. Guo, and S. Sun, "InfiniLoRA: Disaggregated multi-LoRA serving for large language models," *arXiv:2604.07173*, 2026.

[20] Z. Zhu, Q. Su, Y. Ding, K. Song, S. Wang, and G. Pekhimenko, "LoRAFusion: Efficient LoRA fine-tuning for LLMs," in *Proc. EuroSys*, 2026, pp. 586–604.

[21] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Proc. NeurIPS*, 2022.

[22] Z. Ye, L. Chen, R. Lai, W. Lin, Y. Zhang, S. Wang, T. Chen, B. Kasikci, V. Grover, A. Krishnamurthy, and L. Ceze, "FlashInfer: Efficient and customizable attention engine for LLM inference serving," in *Proc. MLSys*, 2025.

[23] P. Whittle, "Restless bandits: Activity allocation in a changing world," *J. Applied Probability*, vol. 25, pp. 287–298, 1988.

[24] R. R. Weber and G. Weiss, "On an index policy for restless bandits," *J. Applied Probability*, vol. 27, no. 3, pp. 637–648, 1990.

[25] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, 2009.